☽ Nightwatch

A dependency-free cybersecurity toolkit — phishing detection, pcap traffic analysis, honeypots and log correlation. Version **0.2.0**

Python: 3.9+ | **Dependencies:** none (standard library only) | **Tests:** 46 (pytest) | **License:** MIT

1. Overview

Nightwatch is a modular cybersecurity toolkit that runs on nothing but the Python standard library. It detects phishing links and emails, analyzes raw packet captures for scanning, C2 beaconing and suspicious traffic, runs decoy honeypot servers that fingerprint attackers, and hunts red flags in server logs — all through a single CLI and an optional browser dashboard.

Modules

Module	What it does
`phishing`	Heuristic URL/email scoring (typosquatting, punycode homographs, URL shorteners, credentials in URL, suspicious TLDs) plus live verification: DNS resolution and redirect-chain following.
`network`	Hand-rolled .pcap parser with TCP flow tracking (handshake completion), HTTP request/response extraction, DNS query extraction and C2 beaconing detection (periodic same-peer traffic).
`honeypot`	TCP and HTTP decoy servers that log every connection and fingerprint attackers (suspicious path probing, brute-force patterns).
`logs`	Collects JSON/JSONL/CSV/syslog/plain-text logs, correlates them with built-in or custom JSON rules, detects bursts and writes markdown reports.
`web`	SOC-style dark dashboard served by a stdlib-only HTTP server. No framework, no build step.

2. Installation

```
pip install .
```

or run directly from the repository root without installing. Development setup:

```
pip install -r requirements-dev.txt
python -m pytest
```

3. Command Line Reference

phish — phishing detection

```
python -m nightwatch.cli phish --url <URL> [--verify] [--json]
python -m nightwatch.cli phish --file <email.txt> [--json]
```

Option	Description
`--url`	URL to score (0–100 risk score).
`--file`	Text file (email body) to scan; every link inside is scored.
`--verify`	Live verification: resolve the domain via DNS, follow the redirect chain manually and report the final destination.
`--json`	Machine-readable output.

Exit codes: `0` = safe, `1` = suspicious/phishing, `2` = usage error — CI-friendly.

pcap — traffic analysis

```
python -m nightwatch.cli pcap <capture.pcap> [--json]
```

Output includes packet/protocol counts, unique and *completed* (fully handshaken) connections, websites visited (HTTP Hosts), user agents, DNS queries, possible C2 beacons and severity-ranked findings.

honeypot — decoy servers

```
python -m nightwatch.cli honeypot --http --port 8080
python -m nightwatch.cli honeypot --port 2222 --respond "220 fake ftp" --out honey.jsonl
```

Option	Description
`--host`	Bind address (default `0.0.0.0`).
`--port`	Listen port (default `8080`).
`--http`	Fake HTTP server instead of raw TCP.
`--respond`	Text to send back to each TCP client (banner).
`--out`	JSONL log file (default `honeypot.jsonl`).

logs — log analysis

```
python -m nightwatch.cli logs <file-or-dir> [--rules <rules.json>] [--out report.md] [--json]
```

Option	Description
`--rules`	JSON file with custom detection rules (see section 6).
`--out`	Write a markdown report to this path.
`--json`	Machine-readable output.

web — dashboard

```
python -m nightwatch.cli web [--host 127.0.0.1] [--port 8000] [--no-browser]
```

> Port 8000 may already be in use on your machine; use `--port 8001` to pick a free port.

4. Web Dashboard

The dashboard is served by a stdlib `ThreadingHTTPServer` — no Flask, no npm, no build step. Open the printed URL in a browser.

Tab	Features
Phishing	Check a link or paste an email. Optional *live verification* checkbox: shows resolved IPs, the full redirect chain, the final destination and a warning when the link crosses domains.
Traffic	Upload a .pcap: packets/connections/established-connection stats, protocols, top sources, destination ports, websites visited, DNS queries, possible C2 beacons and findings.
Honeypot	Start/stop TCP or HTTP decoys, watch live connections every 2 s, clear logs and run attacker fingerprint analysis.
Logs	Upload JSON/JSONL/CSV/syslog/text logs; view findings and bursts; download a markdown report.

HTTP API

```
GET  /api/status                server version + honeypot state
GET  /api/honeypot/status       honeypot running state
GET  /api/honeypot/events       last 200 logged events
GET  /api/honeypot/analysis     attacker fingerprint analysis
POST /api/honeypot/start        {"mode":"http|tcp","host","port","respond"}
POST /api/honeypot/stop
POST /api/honeypot/clear
POST /api/phish                 {"url","verify":true}  → score + verification
POST /api/phish/email           {"text": "..."}
POST /api/pcap                  raw .pcap bytes in body
POST /api/logs?name=app.log     raw log file bytes in body
```

5. Findings Format

All modules report suspicious activity in one unified shape:

```
{
  "severity": "HIGH",      // HIGH | MEDIUM | LOW
  "label": "possible-beacon",
  "count": 6,              // number of involved events
  "detail": "10.0.0.1 -> 203.0.113.9:443 at ~5s intervals"
}
```

Detection labels

Module	Label	Severity	Meaning
network	`syn-scan`	HIGH	A source sent SYN probes to 3+ distinct ports.
network	`icmp-sweep`	MEDIUM	ICMP echo probes to 5+ destinations.
network	`sensitive-port-traffic`	MEDIUM	Traffic to admin/database ports (23, 445, 1433, 3306, 6379, 27017, 3389, 5555, 1337, 5900).
network	`possible-beacon`	HIGH	5+ packets to one peer at near-regular intervals (<30% variance).
honeypot	`sensitive-path-probing`	HIGH	Attacker hit sensitive paths (admin, wp-login, .env, etc.).
honeypot	`brute-force-pattern`	MEDIUM	5+ requests from a single source.
logs	`failed-login`, `sql-injection`, `xss`, `path-traversal`, `privileged-account`, `forbidden-access`, `malware-indicator`, `invalid-user`, `denied`, …	—	Built-in regex rules over normalized log messages (see `DEFAULT_RULES`).

6. Custom Log Rules

Rules are regex patterns with a severity and a label, supplied as a JSON list:

```
[
  {"pattern": "token expired", "severity": "HIGH", "label": "token-expiry"},
  {"pattern": "disk full",     "severity": "LOW",  "label": "disk-space"}
]
```

```
python -m nightwatch.cli logs app.log --rules rules.json --out report.md
```

Rules match case-insensitively against the message. A rule without a `pattern` is rejected with an error.

7. Phishing Scoring

URLs are scored 0–100; verdicts: `safe` (0–39), `suspicious` (40–69), `phishing` (70+). Signals include:

- Brand typosquatting via Levenshtein distance (paypai → paypal, goolge → google)
- Punycode homograph domains (`xn--…`)
- URL shorteners (bit.ly and similar, score 40)
- Credentials embedded in the URL (`paypal.com@evil.io`)
- IP-literal hosts, hyphenated domains, suspicious TLDs
- Phishing keywords in path/host (verify, login, secure, account…)

Live verification (`--verify`) resolves the domain, walks the redirect chain without following it blindly, and flags cross-domain redirects — so a "safe-looking" short link that lands on a different domain is exposed.

8. Project Layout

```
nightwatch/
  __init__.py         version
  cli.py              argparse CLI (phish / pcap / honeypot / logs / web)
  phishing/
    detector.py       heuristics + verify_url (DNS + redirect chain)
  network/
    pcap.py           hand-rolled pcap / Ethernet / IPv4 / TCP / UDP parser
    analyzer.py       flows, handshakes, HTTP/DNS extraction, beaconing
  honeypot/
    server.py         TCP + HTTP decoy servers
    tracker.py        attacker fingerprinting + unified findings
  logs/
    collector.py      JSON / JSONL / CSV / syslog / text parsing
    analyzer.py       DEFAULT_RULES, load_rules(), bursts, markdown reports
  web/
    server.py         stdlib HTTP API (ThreadingHTTPServer)
    static/           index.html, app.js, styles.css (dark SOC UI)
tests/                pytest suite — 46 tests, no network access required
pyproject.toml        packaging; `pip install .` gives the `nightwatch` command
README.md             quick-start guide
```

9. Development

Run the tests

```
python -m pytest tests/ -q
```

Console script

After `pip install .`, a `nightwatch` command is available that behaves identically to `python -m nightwatch.cli`.

10. Changelog

Version	Highlights
0.1.0	Core modules, stdlib web dashboard, heuristic phishing scoring, basic pcap analysis, honeypots, log rules.
0.2.0	TCP flow tracking + handshake completion; HTTP/DNS extraction; C2 beacon detection; live phishing verification; custom JSON log rules; unified `{severity,label,count,detail}` findings;

	pyproject packaging and README.